Xiatech MSc Assignment Brief

Exploring Agentic AI for Retail: the Data Hygiene Watchdog Agent

Background

In this assignment you will build a **Data Hygiene Watchdog**: an autonomous agent that checks a retailer's data every day as it arrives, catches quality problems before anyone relies on them, and fixes them or explains them to a human.

At **Xiatech**, we specialise in **real-time integrations, insights and AI/ML solutions** for some of the world's largest retailers. Our platform, **Xfuze**, gives them a unified view of their operations so they can make smarter, faster decisions.

This assignment is designed to take you out of your comfort zone, as you'll explore the emerging field of **Agentic AI**: AI systems that don't just provide insights but can autonomously plan and execute actions in complex enterprise environments. Every report, forecast and AI agent a retailer runs is only as good as the data underneath it. Rather than waiting for a client to spot a broken number in a dashboard, your agent will catch bad data as it lands, find where it came from and repair it.

You will work in 7 **groups of 5 students** (see list). Each group will have the same use case, and Xiatech will act as your **industry partner**, guiding and evaluating your solutions (see Assessment).

Assignment Objectives

By the end of the assignment, you should be able to:

1. **Explain** the principles of **Agentic AI** and how it differs from traditional AI/ML.
2. **Design and build** a prototype system that integrates:
    - A **PostgreSQL database**, running locally in Docker, storing your retailer's data as it arrives from several source systems.
    - An **agent** built with Google's **Agent Development Kit (ADK)** in Python, calling tools on an **MCP (Model Context Protocol) server** you build with the

MCP Python SDK (version 2.x, whose `MCPServer` class was called FastMCP in 1.x).

- A **Javascript-based UI**, served by a small **REST API** built with FastAPI, to demonstrate and visualise your solution.

3. **Demonstrate autonomy** by having the agent not just report problems but fix them, with a human approval step for the risky fixes.
4. **Prove it in a simulation**, and present your work in a structured presentation deck to Xiatech.

Everything runs on your own machines (except the LLM): there is no cloud service to set up. A free Google account is enough if you use Gemini.

Use Case

The use case focuses on designing an autonomous software agent that **checks every new batch of data, finds what is wrong with it, and repairs it** before it reaches the tables that reports and other agents read.

A retailer's data arrives every day from many systems: tills in the stores (POS), the website, the CRM and the product catalogue (PIM). Each one can break the data in its own way: a connector update stops sending customers' email addresses, the same customer registers twice, a supplier renames a column, a price arrives as £1,999.00 instead of £19.99, or an order points to a product that does not exist. Today these problems are usually found late, often by the client, then fixed by hand and never written up, so the same problems keep coming back.

Your agent does one job: it keeps the data clean. It **never deletes data**: rows it cannot fix, and extra copies of duplicated rows, are moved to quarantine with the reason, and every change it makes can be undone. Another agent at Xiatech watches whether the data pipelines themselves are running, so when a feed did not arrive at all, your agent hands the case over instead of acting itself (imagine there are other agents in the eco-system and what their specs would be).

The goal is to illustrate how data-driven decision-making and **single-task agents working together** can enable near real-time operational automation in retail.

Detailed Workflow

The agent runs once per simulated day, after the day's batches have landed in the staging tables and the simulator has profiled them. The thresholds below are starting values: you may tune them in Week 3 if you can justify the change.

1. **Detection**, per batch
   - Read the profile of each of today's batches (one per source feed): **null rate**, **distinct count**, minimum and maximum, and **format match rate** (for example, the share of emails that look like emails) for every column.

- Each profile comes with its **baseline** null rate and format match: the same column over the last 30 days, learned only from rows that reached the core tables, so problem rows never become part of what counts as normal.
- Run the six checks in the table below, as code. Every failed check becomes an **issue** record.

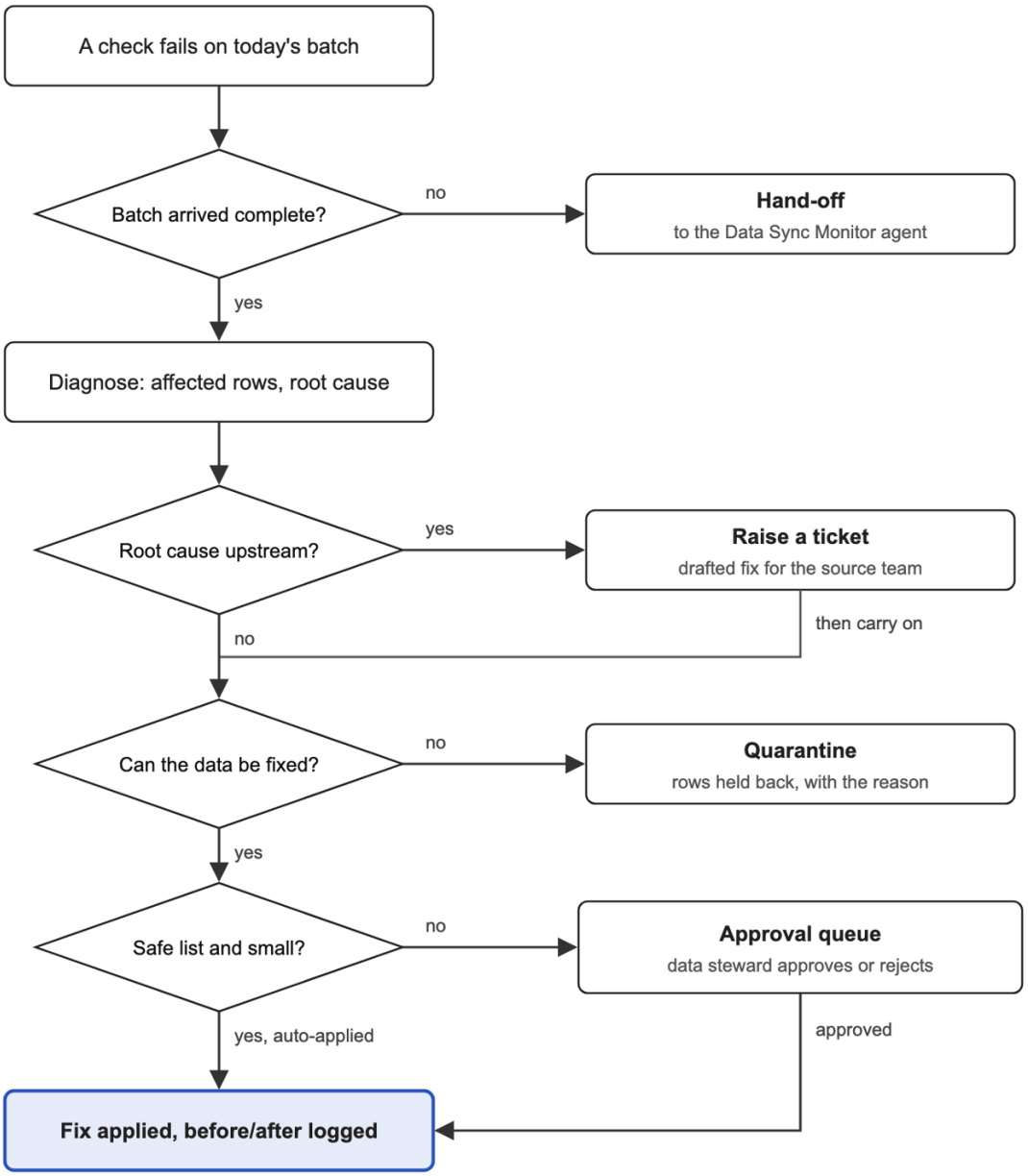
2. **Diagnosis**
   - For each issue, find the affected rows and look at a sample of them.
   - Work out the likely **root cause**: which feed, from which batch it started, and whether the source changed its schema. A cause is **upstream** when a source system or connector changed; otherwise it is **local** to this batch.
   - Group issues that share a root cause, and have the LLM write a short explanation in plain English.
3. **Remediation Workflow**
   - **Ticket**: when the cause is upstream, raise a ticket with the root cause, the number of affected rows, a sample and a **drafted fix** for the team that owns the source. Then carry on: the data in front of you still needs fixing.
   - **Auto-fix**: a fix on the **safe list** that touches **200 rows or fewer** is applied straight away. The safe list holds fixes that cannot lose information: trimming spaces, standardising formats (dates, phone numbers, postcodes, the case of emails), quarantining exact duplicates, mapping a renamed column back to its usual name, and filling a missing value from another source that holds it. A column mapping counts as one fix, however many rows the batch has.
   - **Human approval**: any other fix, or a safe-list fix that touches more than 200 rows, waits in an approval queue for a **data steward**. Examples are merging near-duplicate customers and correcting a price that looks wrong.
   - **Quarantine**: rows that cannot be fixed are moved to quarantine with the reason, so they never reach the core tables.
   - **Hand-off**: if a batch is missing or nearly empty, record a hand-off to the Data Sync Monitor agent and leave the batch alone.
4. **Synchronisation**
   - Every fix is written to the `fixes` table with the value **before and after**, so it can be undone.
   - Each staging row carries a status: `pending`, `clean`, `quarantined` or `awaiting_approval` (then the simulator sets `promoted`, `merged` or `lost`). After your agent has run, the simulator promotes every row still pending or clean.
5. **Feedback Loop**
   - Every decision, with its reasoning, is written to an agent log in PostgreSQL.
   - Rejected fixes and the steward's reasons are passed back into the next day's prompt, so the agent learns from them.
   - An issue that comes back three days in a row (same feed, table, column and check: its `issue_key`) has its ticket escalated.

Starting checks

Check	Rule	Agent action
Completeness	Null rate of a column more than 5 percentage points above its baseline	For a changed customer, keep the value already on file; fill from another source that holds it (web orders carry the customer's email); otherwise quarantine the rows or promote them with the value missing, and raise a ticket
Uniqueness	The same key twice in a batch, or the same customer twice once email and phone are normalised	Exact duplicates: quarantine the extra copies automatically. Near duplicates: propose a merge for approval
Validity	A value fails its format, or a price is over 10× or under 0.1× that SKU's catalogue price	Format problems: standardise automatically. Out-of-range values: propose a correction for approval
Consistency	A row points to something that does not exist (an order for an unknown SKU or customer)	Quarantine the rows; raise a ticket if more than 20 rows are affected
Schema drift	A column is missing, added, renamed or changes type	Map a renamed column automatically when the match is clear; otherwise raise a ticket
Volume	POS and web only: row count more than 30% away from that weekday's average over the last 30 days. CRM and PIM send small, irregular updates, and an empty PIM batch is normal	Missing, or under 20% of that average: hand off to the Data Sync Monitor. Otherwise log a warning

Why quarantine and never delete: a fix that turns out to be wrong must be reversible. A deleted row cannot be checked, corrected or restored; a quarantined row can.

Every failed check goes through the questions below. An upstream cause always gets a ticket, and the data is still fixed when a fix exists. A fix is applied without a human only when it is on the safe list and small:



What the Agent Decides

The six checks are plain code: they produce candidate issues. A script could do that much on its own, so the marks for the agentic part come from the **judgement calls**, which are the agent's job:

- **Grouping** issues that share one root cause into one diagnosis and one ticket.
- **Recognising a renamed column** (is `item_count` really `quantity`?) and deciding whether the match is clear enough to map.
- **Near duplicates**: deciding whether two customer records are the same person, and proposing the merge.
- **Choosing the action**: fill, quarantine, propose a correction, or ticket only, and explaining why.

- **Telling a real event from a problem**: a Black Friday spike listed in the trading calendar, or two people in one household who share a phone number, must not be “fixed”.
- **Drafting tickets** that the source team could act on.
- **Learning from rejections**: reading the steward’s reasons and changing what it proposes the next day.

Starter Kit

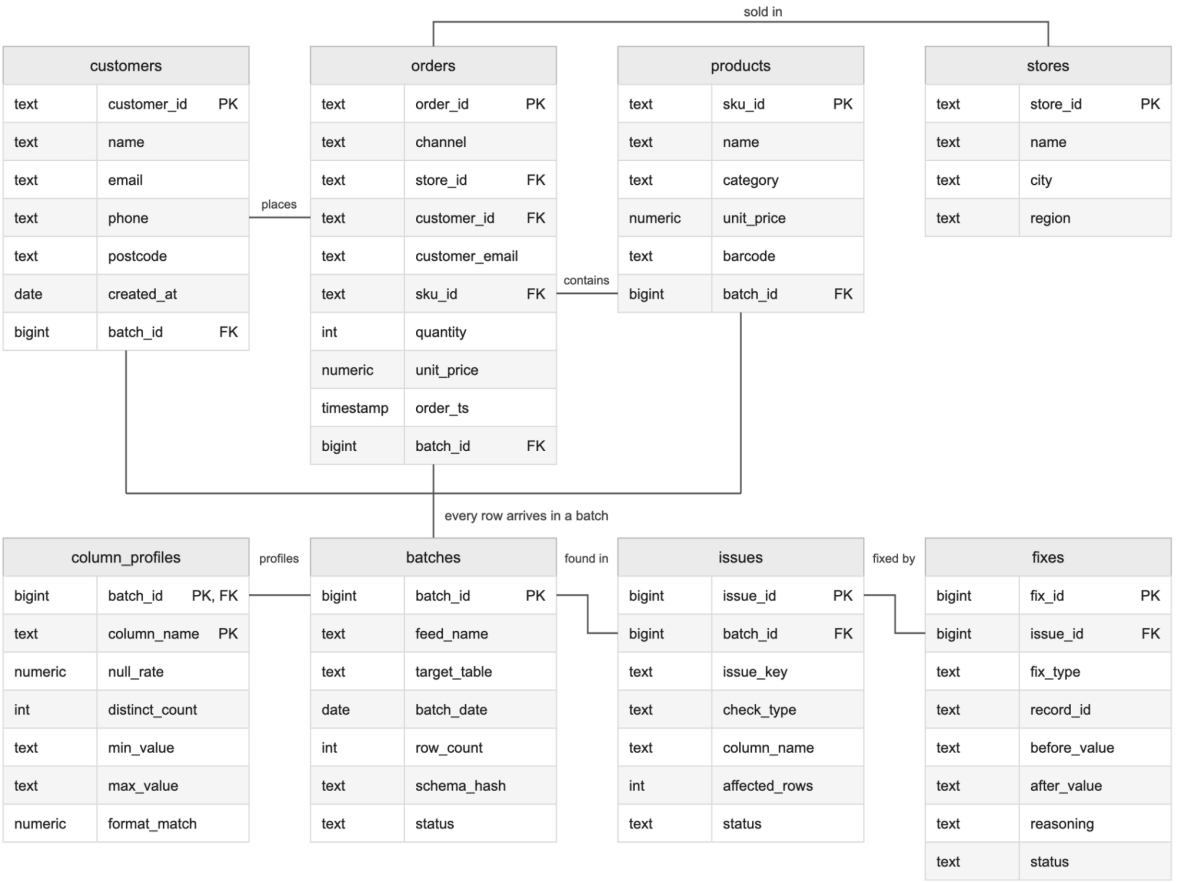
Xiatech provides a starter kit, so you can spend your time on the agent rather than on plumbing. It is at **github.com/albertocalsa/data-hygiene-starter-kit**, and its README explains how to install and run everything:

Component	What it does
`docker-compose.yml`	Starts a local PostgreSQL database. generate_data.py then creates the schema below, the staging tables (raw JSON, one row per record, with a status) and tables for issues, fixes, quarantine, tickets, hand-offs and the agent log.
`generate_data.py`	Creates the synthetic retailer: about 5,000 customers, 300 products, 20 stores and 90 days of clean history, arriving through four feeds (POS, web, CRM and PIM). Web orders carry the email typed at checkout.
`scenario.py`, `scenarios/`	Ready-made scenarios (`dev`, `practice`, `chaos`): the 28 simulation days with data problems planted in them. `python scenario.py dev` chooses one. Where the problems are, and the answers the simulator scores with, stay hidden.
`profiler.py`	Profiles every batch: null rate, distinct count, minimum and maximum, and format match for each column (ids, prices, dates, emails, UK postcodes and UK phone numbers), with the baseline. The simulator runs it; your agent reads the results.
`simulator.py`	Runs the simulation one day at a time (`step`) or all 28 days (`run`), calling your agent once a day, playing the data steward and writing a results report.
`hello_mcp.py`, `hello_agent.py`	A one-tool MCP server and an ADK agent that calls it: a working starting point.

Synthetic Data

Once the generator has run, PostgreSQL holds the tables below. New batches land in staging tables (`staging_orders`, `staging_customers`, `staging_products`) as raw

JSON, exactly as delivered, so a renamed column or a £ sign in a price still lands and can be inspected. Explore them before you write any agent code:



The kit also writes `data/calendar.csv`, the trading events the retailer knows in advance (such as Black Friday), so your agent can tell an expected spike from a problem.

The kit also creates tables for quarantine, tickets, hand-offs and the agent log (every decision with its reasoning, to be displayed in the UI). You may add columns or tables of your own (create them as the `watchdog` user and grant them to `agent`).

How the Simulation Works

Your agent is an async Python function that the simulator imports: `async def run_day(day)`. It runs in its own process, logged in to the database as the restricted user `agent`. Each simulated day runs the same six steps, in this order:

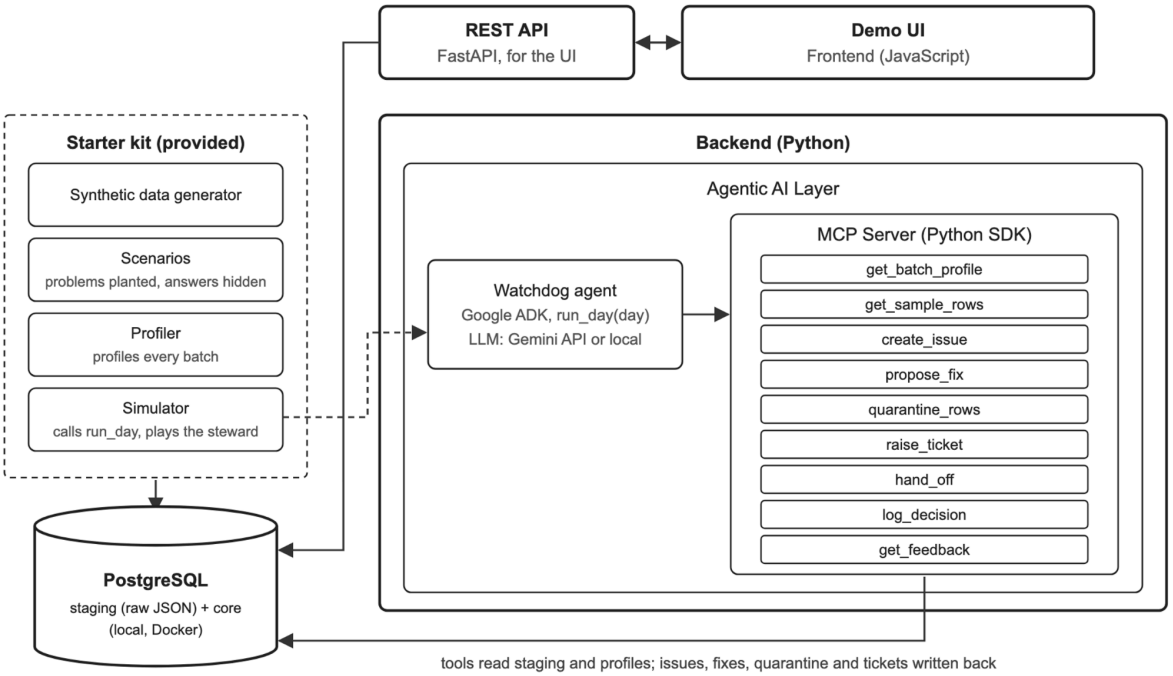
1. **Morning.** The simulator plays the **data steward**: it approves the fixes awaiting approval from earlier days that are correct, and rejects the wrong ones with a reason your agent can read. Approved fixes are applied to their staging rows.
2. Each feed delivers today's batch into the staging tables. On some days the scenario has planted problems in it, and some days a feed does not arrive at all.
3. The simulator profiles every new batch with `profiler.py` and stores the results in `column_profiles`.

4. Your agent runs once and checks, fixes, quarantines, raises tickets and hands off through its MCP tools.
5. The simulator promotes every staging row that is not quarantined or awaiting approval, converting it to the core tables' types. Rows that fail the conversion or point to something missing are recorded as lost.
6. The simulator adds a line to its results report. A gated fix therefore keeps bad data out of the core tables until the next morning; after day 28 one last morning runs.

The **results report** shows, for each day: problems caught and missed, fixes right and wrong (a fix the steward rejects counts as wrong), false alarms, and **data health** (the share of the day's rows that reached the core tables with their true values). The report is for you to learn from, not a ranking. Because the answers stay inside the scenario, your agent must judge from the data alone, exactly as a real data team would. In the final session Xiatech runs your agent on a **scenario you have not seen** (the same kinds of problem on other days and feeds, with other counts and variants), so build it to read the data, not to remember the 28 days.

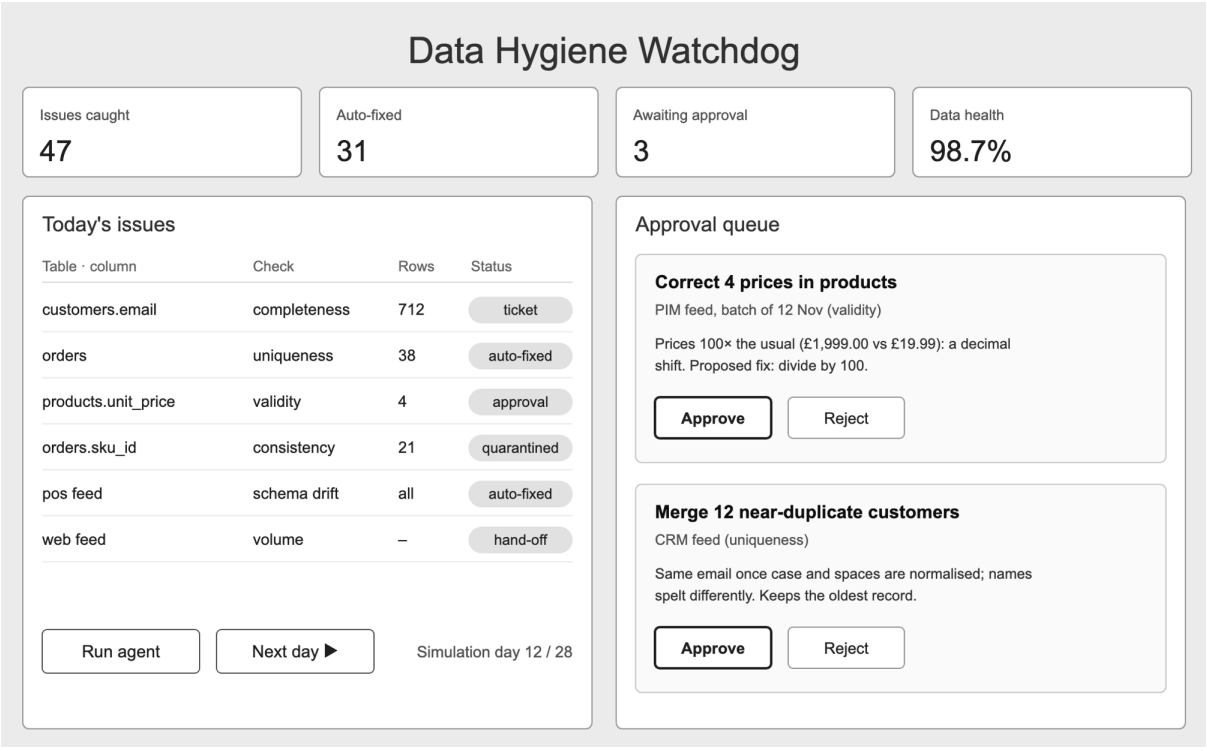
The complete system architecture will end up looking something like this:

- The starter kit loads the synthetic data into PostgreSQL and provides the scenarios, the profiler and the simulator.
- The simulator delivers and profiles each day's batches, then calls your agent's `run_day`.
- The **Watchdog agent**, built with **Google ADK**, decides what is wrong, how to fix it and whether a human must approve, and explains why. It acts only through the tools on your **MCP server**, which it connects to with ADK's `McpToolset`; the tools read and write PostgreSQL through a restricted database login (`agent`), which cannot see the simulator's results. Treat the nine tools shown as a minimum, and add what you need.
- The **LLM**: we recommend **Gemini Flash** through a free Google AI Studio API key. The free tier limits requests per minute and per day, so add retries with a pause, and do not re-run all 28 days more often than you need to. A local model through Ollama also works if it handles tool calls well (for example `qwen3:4b`, which needs 8 GB of RAM or more); small models such as qwen3:1.7b or Mistral 7B often call tools wrongly.
- A **REST API** (FastAPI) serves the **JavaScript frontend**: issues, fixes, approvals, undo, simulation steps and the results report.



Javascript Frontend

A simple UI to showcase how the agent works. Here is an example of a possible UI with the basic features. Adding other features (for example a chart of each column's null rate over time, or a chat panel to ask the agent why it chose a fix) will be hugely beneficial:



At minimum it should include:

- **Issues view**: today's issues by table and column, with the check that failed, the rows affected and their status.
- **Fix log**: every fix with its before and after values and the agent's reasoning, and a button to undo it.
- **Approval queue**: approve or reject the fixes that need a human, with an optional rejection reason that the agent reads the next day.
- **Simulation controls**: run the agent, step to the next day, and see the current day.
- **KPI tiles**: issues caught, issues auto-fixed and fixes awaiting approval, plus data health from the simulator's results report (the agent itself cannot know what it missed).

Weekly Plan

Week 1: Setup, Data & Simulation

Learning Objectives

- Install the core components step by step: Docker, PostgreSQL, Python, Node.js and related tools.
- Understand database schema design for data arriving in batches from several source systems.
- Learn how data quality is measured (completeness, uniqueness, validity, consistency) and what the simulator reports.
- Practise SQL and data manipulation from Python.

Practical Tasks

1. **Environment Setup**
    - Follow the starter kit README: start PostgreSQL, run `generate_data.py` and `scenario.py dev`.
    - Install Python 3.13 (3.11 to 3.14 also work), set up a virtual environment, and install the kit's requirements (psycopg, google-adk, mcp), plus fastapi and pandas.
    - Verify database connectivity and perform simple queries.
2. **Data Exploration**
    - Step through the first days with the simulator, look at `column_profiles`, and write one SQL query for each of the six checks.
    - Find a few planted problems by hand in the first days: they are the kinds of case your agent must catch.
3. **Scaffolding**

    - Run `python hello_mcp.py --test 1` and `python hello_agent.py 1` from the kit, and create the UI app (React or another JS library) and the REST API skeleton.
4. **LLM Setup**
    - Create a free Gemini API key in Google AI Studio, or run a tool-capable model locally through Ollama (for example `qwen3:4b`).
    - Experiment with simple reasoning prompts and a single tool call.
5. **Baseline Run**
    - Run `simulator.py` with an agent that does nothing, and keep its results report: it shows how much bad data reaches the core tables without your agent.

Deliverables

- Working local PostgreSQL loaded with the starter data.
- SQL queries for the six checks, and the do-nothing results report.
- Mini technical note summarising the data model and the checks.
- LLM accessible and responsive

Week 2: The Agent, MCP Server & Frontend UI

Learning Objectives

- Build an agent with Google ADK that reasons over data and acts through tools.
- Expose your data and actions as tools on an MCP server built with the MCP Python SDK (`MCPServer`, formerly FastMCP).
- Develop a simple React UI (or other JS library), with its REST API, to visualise issues, approve fixes and trigger the agent.
- Understand how the agent, the MCP server, the REST API, PostgreSQL and the UI talk to each other.

Practical Tasks

1. **MCP Server**
    - Implement the tools: `get_batch_profile`, `get_sample_rows`, `create_issue`, `propose_fix`, `quarantine_rows`, `raise_ticket`, `hand_off`, `log_decision` and `get_feedback`.
    - Apply the safe list and the 200-row rule inside `propose_fix`, and record before and after values there, so the agent cannot bypass them: the tool applies a fix straight away or queues it for approval.
2. **The ADK Agent**

- Write the agent's instructions: its single job, the checks, the judgement calls it owns, when not to fix, and that it never deletes data.
- Expose it as `run_day(day)`, connect it to your MCP server, and run it for one simulated day.

3. **Frontend Development (React UI or other JS library or framework)**
   - Build the issues view, the fix log and the approval queue on top of your REST API.
   - Add buttons to run the agent and step to the next day.
   - Use www.xiatech.io for the website styling CSS

Deliverables

- A working ADK agent acting through your MCP server.
- Functional UI for data visualisation, approvals and agent interaction.
- Screenshot walkthrough of one simulated day.

Week 3: Simulation, Tuning & Presentation

Learning Objectives

- Evaluate an autonomous agent over many days, not one.
- Improve decisions using evidence from the simulation.
- Produce documentation, slides, and a demonstration presentation.

Practical Tasks

1. **Full Simulation**

○ Run all 28 days and compare your results report with the do-nothing one.

○ Study the days where a problem slipped through, or where the agent changed something that was not broken.

2. **Tuning**

○ Tune thresholds per table and column, group issues that share a root cause into one ticket, and make the agent act on the steward's rejection reasons.

○ Make sure the agent never stops halfway through a simulation: it must be ready to show how it works and resolves issues on a scenario it has not seen.

3. **Documentation & Presentation**

○ Prepare a final project report and a presentation deck.

○ Include diagrams, your tool definitions, the agent's instructions and your results report.

Deliverables

- A fully functional autonomous agent that completes a 28-day simulation unattended.
- Results report and analysis, compared with the do-nothing run.
- Final documentation and slides ready for presentation.

Bonus: Going Further

Optional — for extra marks

Practical Tasks

1. **A Reviewer Agent**
   - Add a second agent that plays the data steward: it reviews each gated fix against a sample of the rows and similar past decisions, and approves or rejects it with a reason.
2. **Smarter Detection**
   - Go beyond fixed thresholds (for example with seasonal baselines or an anomaly detection model) and show whether it improves your results report.
3. **Chaotic Scenarios**
   - Choose the chaos scenario (`python scenario.py chaos`, then `python reset_sim.py`): a feed that sends yesterday's file again, till clocks left on summer time (sales after the stores close), and two problems hitting the same column on the same day.
4. **Agentic System KPIs**
   - Employee-hours saved (assume a data engineer needs 30 to 60 minutes to find, diagnose and write up one issue).
   - Agent operating cost (approximate LLM usage; assumed figures are fine).
   - Decision quality (share of problems caught before they reached the core tables, and fixes that had to be undone).
5. **Composability (stretch)**
   - Build a simple Data Sync Monitor agent yourselves, as a stretch target of this assignment, that picks up your hand-offs, and show the two agents working in sequence.

By the end of this assignment, you will have hands-on experience building an **end-to-end Agentic AI system** that integrates data, tools and automation, with a responsive frontend, and proven in a realistic simulation. All these skills are directly relevant to the future of enterprise AI in retail.

Assessment

Part	What we look for
Working system	The agent completes a simulation unattended, acting only through its MCP tools, with a usable UI
Decision quality	The results report on the unseen scenario in the final session, and how well you explain what the agent missed
Agentic design	The judgement calls the agent makes, its instructions and tools, and how it uses the steward's rejections
Report	The principles of Agentic AI, your design and your results (details will be provided)
Presentation	A 10–15 minute demonstration and deck (details will be provided)